

Netaji Subash University of Technology

New Delhi - 110031



Digital Signal Processing

EAEP502

Practical Record File

2026-2027

Submitted By:

Name: Yatharth Shahi

Roll No: 2024UEA6638

Branch: ECAM

Section: 2

Submitted To:

Dr. Rashmi Gupta

Electronics &
Communication
Dept.

Output:

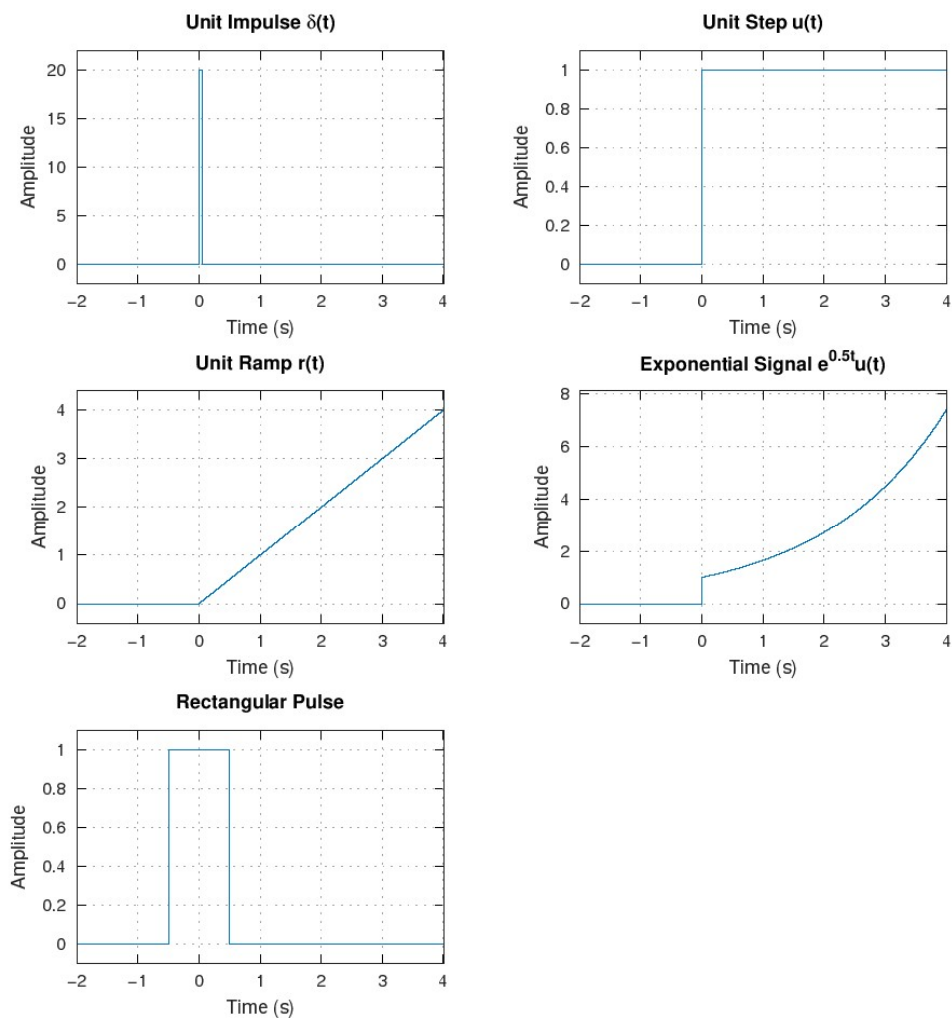


Fig. 1.1: Singularity, exponential and rectangular-pulse signals

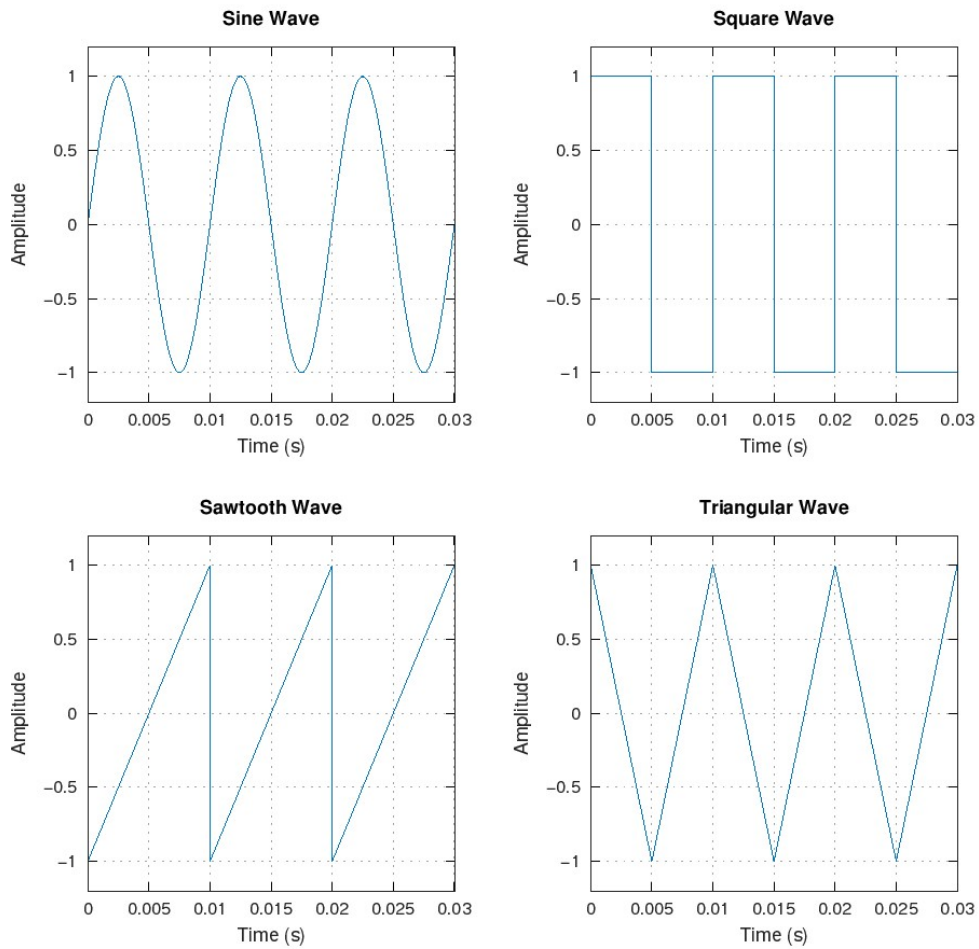


Fig. 1.2: Periodic continuous-time signals

Result:

The basic continuous-time signals (unit impulse, unit step, unit ramp, exponential, rectangular pulse, and sine, square, sawtooth and triangular waves) were generated and plotted using MATLAB.

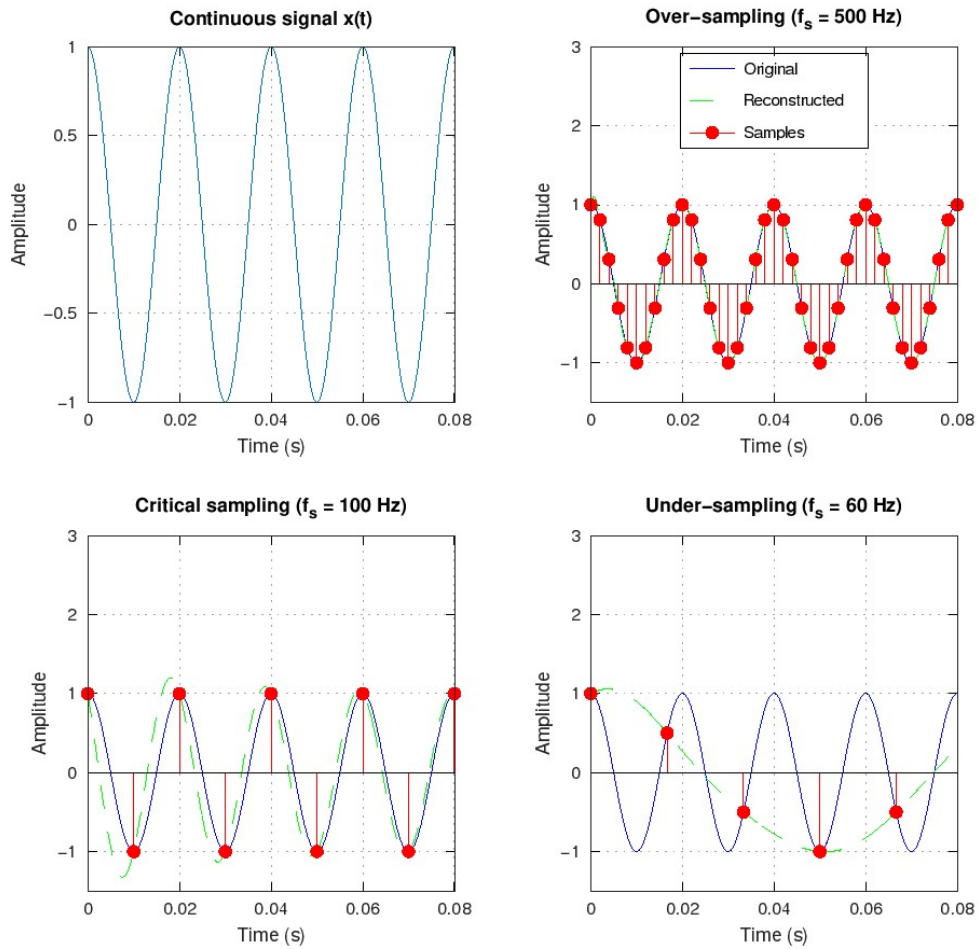


Fig. 2.1: Original, sampled and reconstructed signals for different sampling rates

Result:

The sampling theorem was verified. For $f_s = 500$ Hz and 100 Hz ($f_s \geq 2f_m$) the 50 Hz signal was reconstructed correctly (MSE 0.00000 and 0.01674 respectively), whereas at $f_s = 60$ Hz ($< 2f_m$) aliasing occurred, the MSE rose to 1.00849 and the signal appeared at 10 Hz.

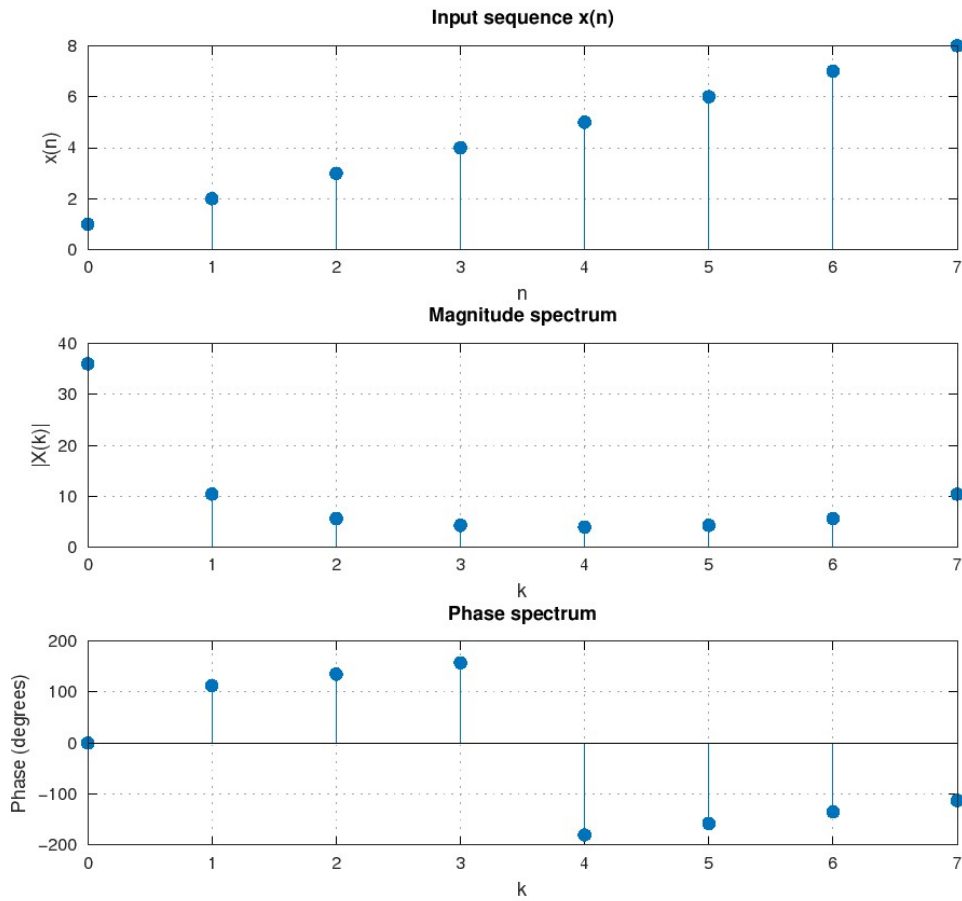


Fig. 3.1: 8-point DFT: input sequence, magnitude spectrum and phase spectrum

Result:

The 8-point DFT of $x(n) = \{1, 2, 3, 4, 5, 6, 7, 8\}$ was computed using the twiddle-factor matrix and its magnitude and phase spectra were plotted: $X(0) = 36$ and $X(1) = -4 + j9.6569$. The results agreed with the built-in fft (difference below 10^{-12}).

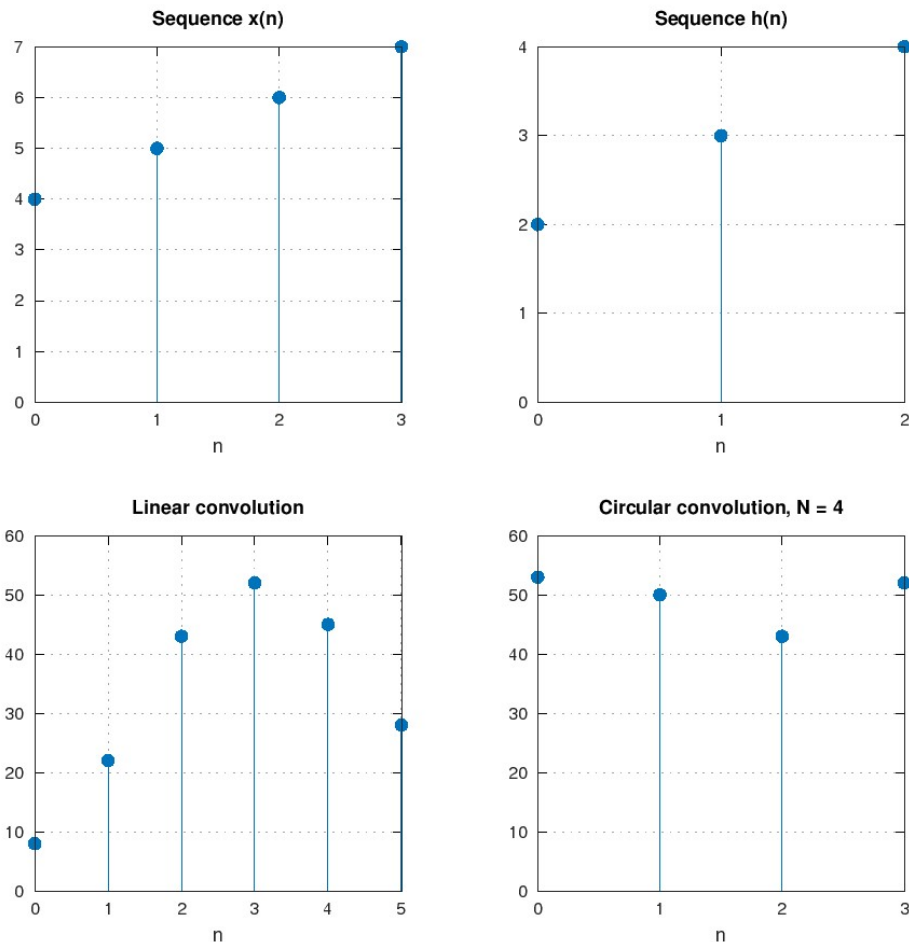


Fig. 4.1: Linear and circular convolution of $x(n) = \{4, 5, 6, 7\}$ and $h(n) = \{2, 3, 4\}$

Output 2: $x(n) = \{4, 3, 2, 1\}$, $h(n) = \{1, 2, 3, 4\}$, $N = 4$

MATLAB Command Window

```
>> exp4_convolution
Enter the sequence x(n) = [4 3 2 1]
Enter the sequence h(n) = [1 2 3 4]
Enter the length N of the circular convolution = 4

Linear convolution           = [ 4 11 20 30 20 11 4 ]
Circular convolution, N = 4  = [ 24 22 24 30 ]
Circular convolution, N = 7  = [ 4 11 20 30 20 11 4 ]
```

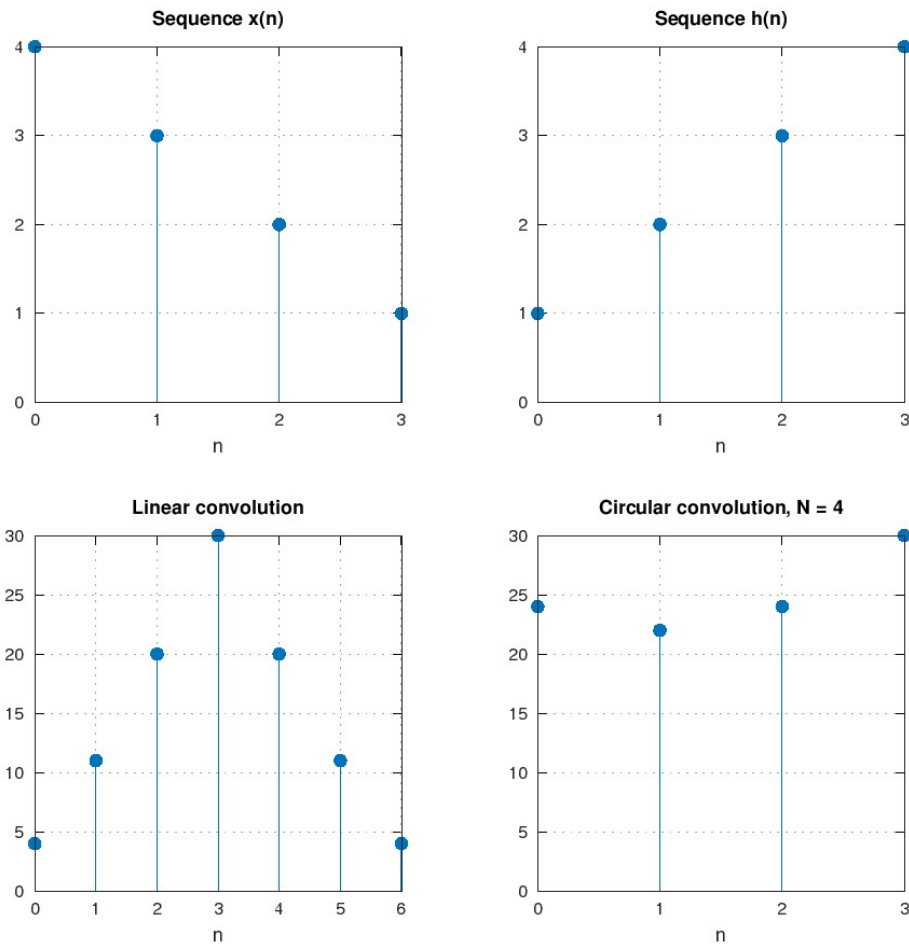


Fig. 4.2: Linear and circular convolution of $x(n) = \{4, 3, 2, 1\}$ and $h(n) = \{1, 2, 3, 4\}$

Result:

Linear and circular convolution of the given sequences were performed. For $x(n) = \{4, 5, 6, 7\}$ and $h(n) = \{2, 3, 4\}$ the linear convolution was $\{8, 22, 43, 52, 45, 28\}$ and the 4-point circular convolution was $\{53, 50, 43, 52\}$; for $x(n) = \{4, 3, 2, 1\}$ and $h(n) = \{1, 2, 3, 4\}$ the 4-point circular convolution was $\{24, 22, 24, 30\}$. Circular convolution with $N = L_x + L_h - 1$ was identical to the linear convolution.

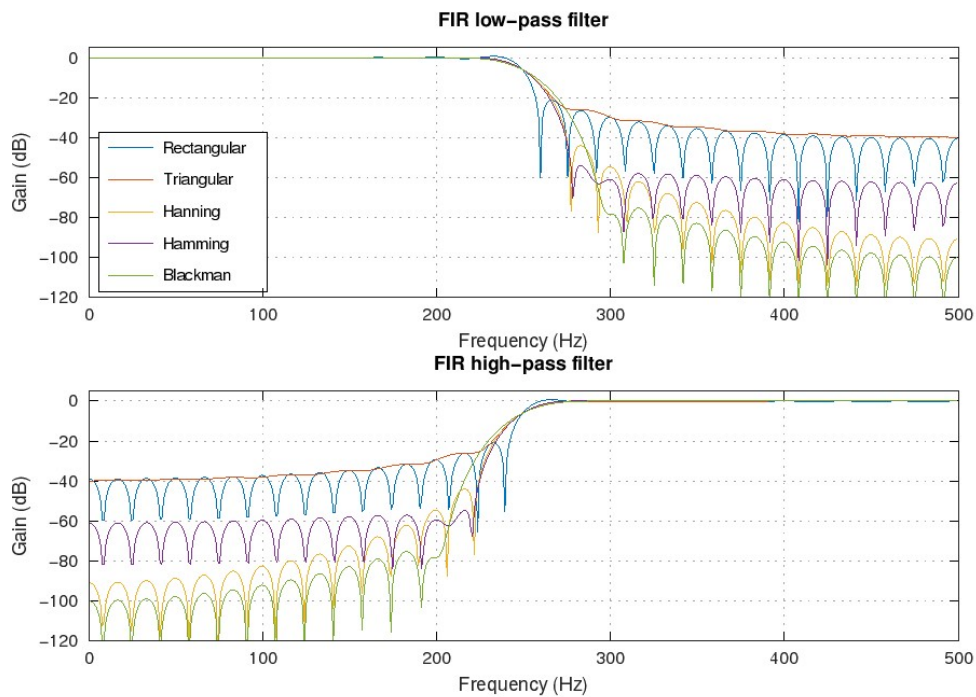


Fig. 5.1: Magnitude responses of the FIR low-pass and high-pass filters designed with different windows

Result:

FIR low-pass and high-pass filters were designed using five different windows. The stop-band attenuation obtained was 30.05 dB (rectangular), 30.02 dB (triangular), 54.87 dB (Hanning), 57.99 dB (Hamming) and 75.26 dB (Blackman), and the pass-band ripple fell from 0.335 dB (rectangular) to 0.002 dB (Blackman).

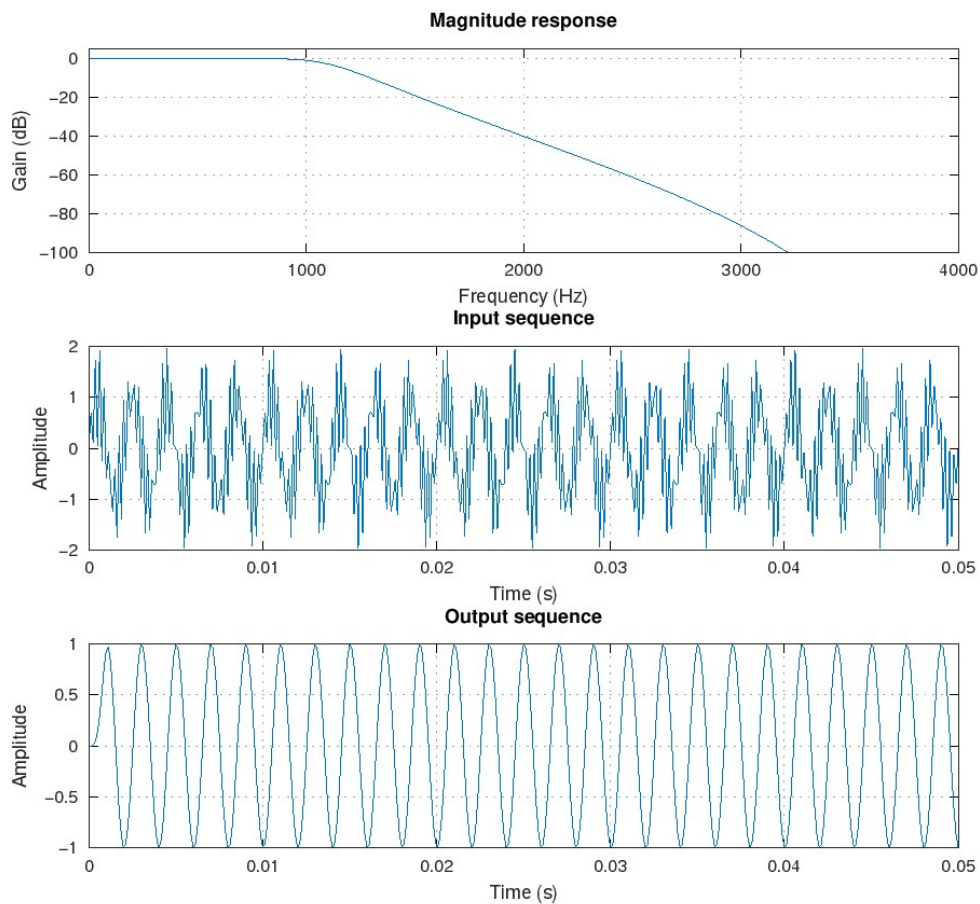


Fig. 6.1: Magnitude response of the Butterworth low-pass filter and filtering of the test sequence

Output 2: Butterworth high-pass filter

MATLAB Command Window

```
>> exp6_iir_filters
Enter choice of filter 1. LPF 2. HPF : 2
Enter the pass-band ripple Rp (dB) = 1
Enter the stop-band attenuation Rs (dB) = 40
Enter the pass-band edge frequency (Hz) = 2000
Enter the stop-band edge frequency (Hz) = 1000
Enter the sampling frequency (Hz) = 8000
```

```
Order N = 6, cut-off = 1856.9 Hz
Gain at 2000 Hz = -1.00 dB
Gain at 1000 Hz = -40.07 dB
```

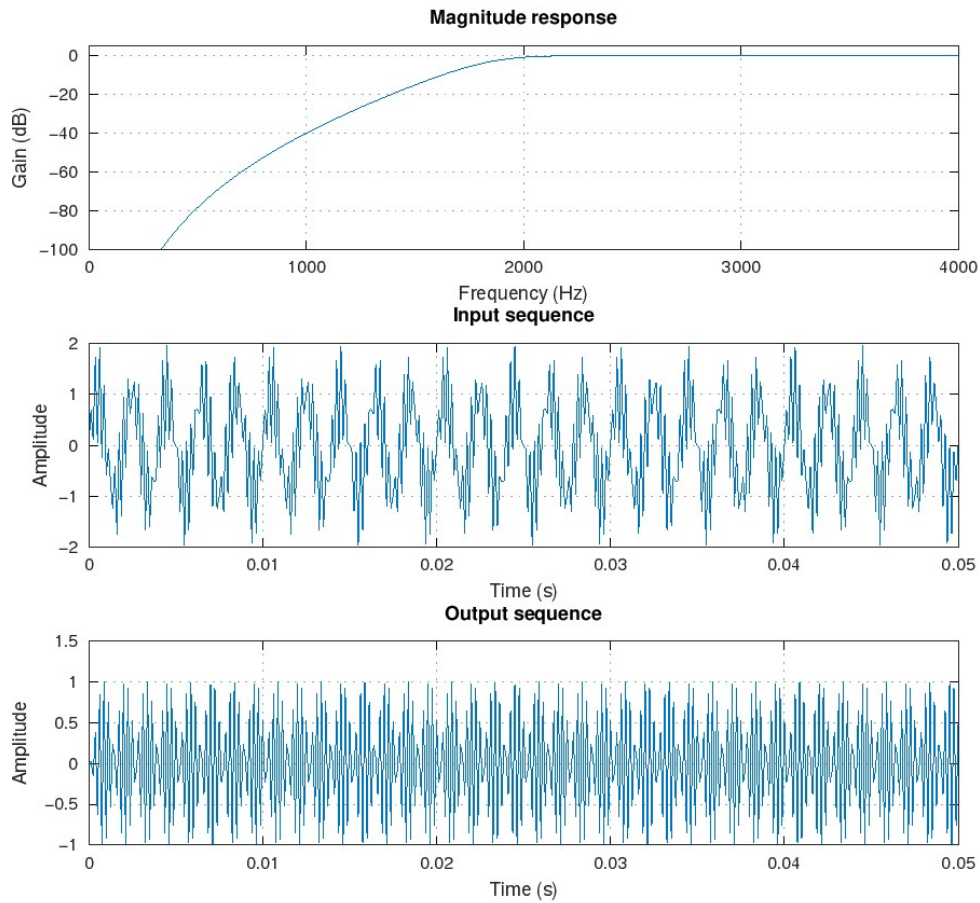


Fig. 6.2: Magnitude response of the Butterworth high-pass filter and filtering of the test sequence

Result:

A sixth-order Butterworth low-pass filter and a sixth-order Butterworth high-pass filter were designed for $R_p = 1$ dB and $R_s = 40$ dB at $f_s = 8000$ Hz. Both met the specification: the gain at the pass-band edge was not below -1 dB and the gain at the stop-band edge was not above -40 dB. The low-pass filter removed the 3600 Hz tone from the test sequence, while the high-pass filter removed the 500 Hz tone.

Procedure:

matlab

```
clc; clear; close all;
fs = 1000; t = 0:1/fs:1-1/fs; N = length(t);
x = cos(2*pi*50*t);
xh = imag(hilbert(x));
fprintf('Max error between hilbert(x) and sin(2*pi*50*t) = %.2e\n', max(abs(xh -
sin(2*pi*50*t))));

H = zeros(1, N);
H(2:N/2) = -1j;
H(N/2+2:N) = 1j;

subplot(2,1,1); plot(t(1:100), x(1:100), 'b', t(1:100), xh(1:100), 'r', 'LineWidth', 1.5);
title('x(t) and its Hilbert transform'); xlabel('Time (s)'); ylabel('Amplitude');
legend('x(t) = cos', 'H[x(t)] = sin'); grid on;
subplot(2,1,2); plot((-N/2:N/2-1)*fs/N, fftshift(angle(H))*180/pi, 'LineWidth', 1.5);
title('Phase of Hilbert transformer H(f)'); xlabel('Frequency (Hz)'); ylabel('Phase (degrees)');
ylim([-120 120]); grid on;
```

Output:

Command Window

MATLAB Command Window

```
>> exp7_hilbert_transform
Max error between hilbert(x) and sin(2*pi*50*t) = 4.82e-14
```

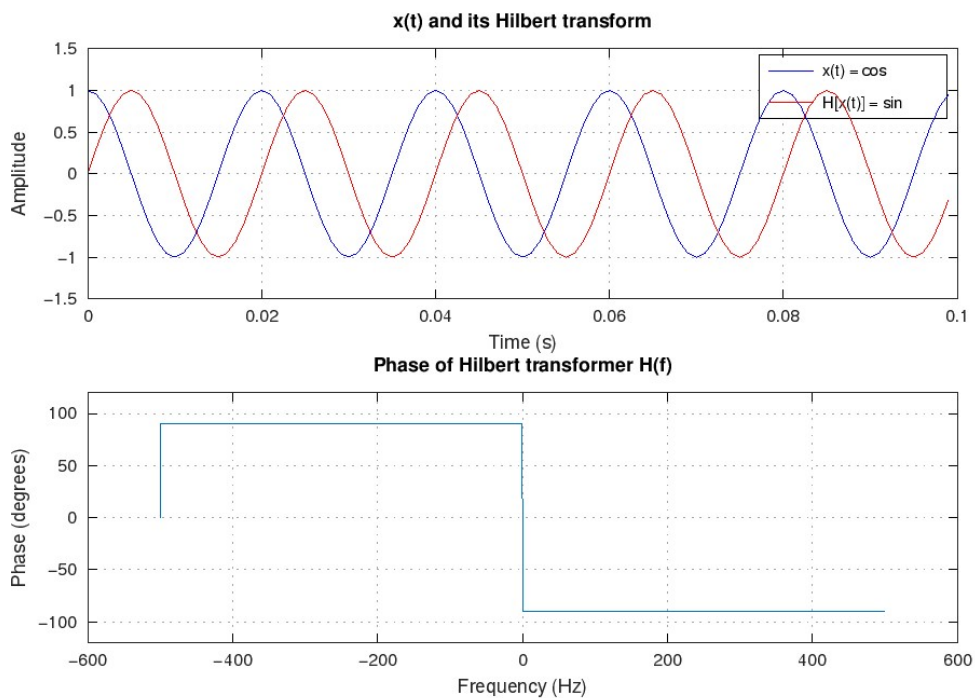


Fig. 7.1: Hilbert transform of $\cos(2\pi \cdot 50t)$ and phase of the Hilbert transformer

Result:

The Hilbert transform of $\cos(2\pi \cdot 50t)$ was obtained as $\sin(2\pi \cdot 50t)$ using the built-in hilbert function (error of the order of 10^{-14}), and the Hilbert transformer showed a -90° phase shift for positive frequencies and $+90^\circ$ for negative frequencies.